

DP Computer Science: B 4.1.5 Construct and Apply Sets as an ADT (Python) (HL Only)

1. Core Concepts & Learning Objectives

Overview

This resource provides four complete one-hour lessons for teaching IB Computer Science topic **B4.1.5, Sets as an ADT**, using Python. Students will master set characteristics, union, intersection, difference, subset relationships, and element removal with `.remove()` vs. `.discard()`.

Key Vocabulary

Term	Definition
Set	An unordered collection of unique elements
Element	An individual item in a set
Union	All elements from both sets ($A \cup B$)
Intersection	Elements common to both sets ($A \cap B$)
Difference	Elements in the first set but not the second ($A - B$)
Subset	All elements of set A are also in set B ($A \subseteq B$)
Superset	Set B contains all elements of set A ($B \supseteq A$)
Proper Subset	$A \subseteq B$ and $A \neq B$ ($A \subset B$)
Membership	Checking if an element exists in a set (<code>in</code>)
Venn Diagram	A visual representation of sets and their relationships

ATL Skills Focus

Skill	Description
Thinking (Conceptual Understanding)	Connecting mathematical concepts to programming
Thinking (Logical Reasoning)	Applying set theory logic to concrete data
Communication	Using precise terminology; interpreting Venn diagrams
Self-Management	Organising steps to solve multi-part problems

2. Lesson 1: Introduction to Sets and Their Characteristics

Learning Intentions

By the end of this lesson, students will be able to:

- Define a "set" as an abstract data type (ADT)
- Explain the two fundamental characteristics of sets: **unordered** and **unique elements**
- **Construct** a set in Python
- Write code to **add** an element to a set and **check** if an element exists in a set

The Bag Analogy

"A set is like a bag of unique Scrabble tiles. You can't have two of the same tile, and the order doesn't matter."

Set Characteristics

text

SET CHARACTERISTICS

UNORDERED

- Elements have no index position
- Cannot access with ``my_set[0]``
- Order is not preserved

UNIQUE ELEMENTS

- No duplicates allowed
- Adding an existing element has no effect
- Useful for removing duplicates from a list

Sets in Python

python

```
# Creating a set from a list (removes duplicates)
my_list = [1, 2, 2, 3, 3, 3, 4]
my_set = set(my_list)
print(my_set) # Output: {1, 2, 3, 4} (duplicates removed)

# Creating a set with curly braces
fruits = {"apple", "banana", "cherry"}
print(fruits) # Output: {'banana', 'apple', 'cherry'} (order not guaranteed)

# Creating an empty set (must use set(), not {})
empty_set = set()
# empty_set = {} # ❌ This creates a dictionary, NOT a set!

# Adding elements
fruits.add("orange")
print(fruits) # Output: {'banana', 'apple', 'cherry', 'orange'}

# Adding a duplicate has no effect
```

```

fruits.add("apple")
print(fruits) # Output: {'banana', 'apple', 'cherry', 'orange'} (unchanged)

# Checking membership
if "apple" in fruits:
    print("Apple is in the set!")

# Trying to access by index causes an error
# fruits[0] # ❌ TypeError: 'set' object is not subscriptable

```

Key Points

Concept	Python Code	Notes
Create set	<code>my_set = {1, 2, 3}</code>	Curly braces
Create empty set	<code>my_set = set()</code>	Not <code>{}</code> (that's a dict)
Add element	<code>my_set.add(4)</code>	No effect if already present
Check membership	<code>if 4 in my_set:</code>	Returns <code>True</code> or <code>False</code>
Remove duplicates	<code>my_set = set(list_with_duplicates)</code>	Converts list to set

3. Lesson 2: Core Set Operations — Union, Intersection, Difference

Learning Intentions

By the end of this lesson, students will be able to:

- Define and visualise **Union, Intersection, and Difference** using Venn diagrams
- Write Python code to perform these three operations using both **operators** and **methods**
- Apply these operations to solve simple logical problems

Venn Diagrams

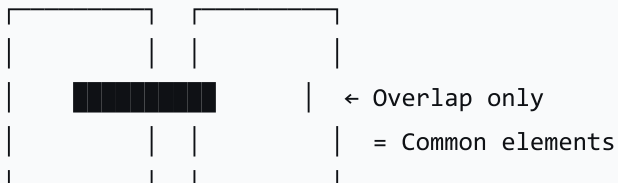
text

VENN DIAGRAMS

UNION ($A \cup B$)



INTERSECTION ($A \cap B$)



DIFFERENCE ($A - B$)



Set Operations in Python

python

```
# Define two sets
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}

# UNION: ALL elements from both sets
union_operator = A | B          # {1, 2, 3, 4, 5, 6}
union_method = A.union(B)      # {1, 2, 3, 4, 5, 6}

# INTERSECTION: Elements common to both sets
intersection_operator = A & B   # {3, 4}
intersection_method = A.intersection(B) # {3, 4}

# DIFFERENCE: Elements in A but not in B
difference_operator = A - B     # {1, 2}
difference_method = A.difference(B) # {1, 2}

# Note: B - A would be {5, 6} (order matters!)
```

Example Application: School Clubs

python

```
# Student data
chess_club = {"Alice", "Bob", "Charlie", "Diana"}
debate_club = {"Charlie", "Diana", "Eve", "Frank"}

# Students in either club (union)
either_club = chess_club | debate_club
print(f"Students in either club: {either_club}")

# Students in both clubs (intersection)
both_clubs = chess_club & debate_club
print(f"Students in both clubs: {both_club}")

# Students only in chess club (difference)
only_chess = chess_club - debate_club
print(f"Students only in chess: {only_chess}")

# Students only in debate club
only_debate = debate_club - chess_club
print(f"Students only in debate: {only_debate}")
```

4. Lesson 3: Subsets, Supersets, and Element Removal

Learning Intentions

By the end of this lesson, students will be able to:

- Define **subset** and **superset** and visualise the relationship
- Write Python code to check for subset and superset relationships
- Explain the difference between the `.remove()` and `.discard()` methods
- Write code to remove elements from a set

Subsets and Supersets

text

SUBSETS AND SUPERSETS

A IS A SUBSET OF B

B

A

All elements of A are in B

$A \subseteq B$

Proper Subset: $A \subseteq B$ and $A \neq B$ ($A \subset B$)

Superset: $B \supseteq A$ (B contains all elements of A)

Subset/Superset in Python

python

```
A = {1, 2}
B = {1, 2, 3, 4}
C = {1, 2}

# Subset (<= and .issubset())
print(A <= B)          # True (A is a subset of B)
print(A.issubset(B))   # True
print(B <= B)          # True (set is a subset of itself)

# Proper Subset (<)
print(A < B)           # True (A is a proper subset of B)
print(B < B)           # False (not a proper subset of itself)

# Superset (>= and .issuperset())
print(B >= A)          # True (B is a superset of A)
print(B.issuperset(A)) # True
```

Removing Elements: .remove() vs .discard()

python

```
my_set = {1, 2, 3, 4, 5}

# .remove() - removes if present; raises KeyError if not found
my_set.remove(3)
print(my_set) # {1, 2, 4, 5}

# my_set.remove(10) # ❌ KeyError: 10

# .discard() - removes if present; does nothing if not found
my_set.discard(4)
print(my_set) # {1, 2, 5}

my_set.discard(10) # ✅ No error
print(my_set) # {1, 2, 5} (unchanged)
```


When to Use Each

text

`.remove()` vs `.discard()`

`.remove()`

Use when: The item **MUST** exist

If not found: Raises `KeyError`

Use case: Critical operations

`.discard()`

Use when: It's okay if the item doesn't exist

If not found: Does nothing

Use case: Non-critical cleanup operations

5. Lesson 4: Applying Sets to Solve Problems

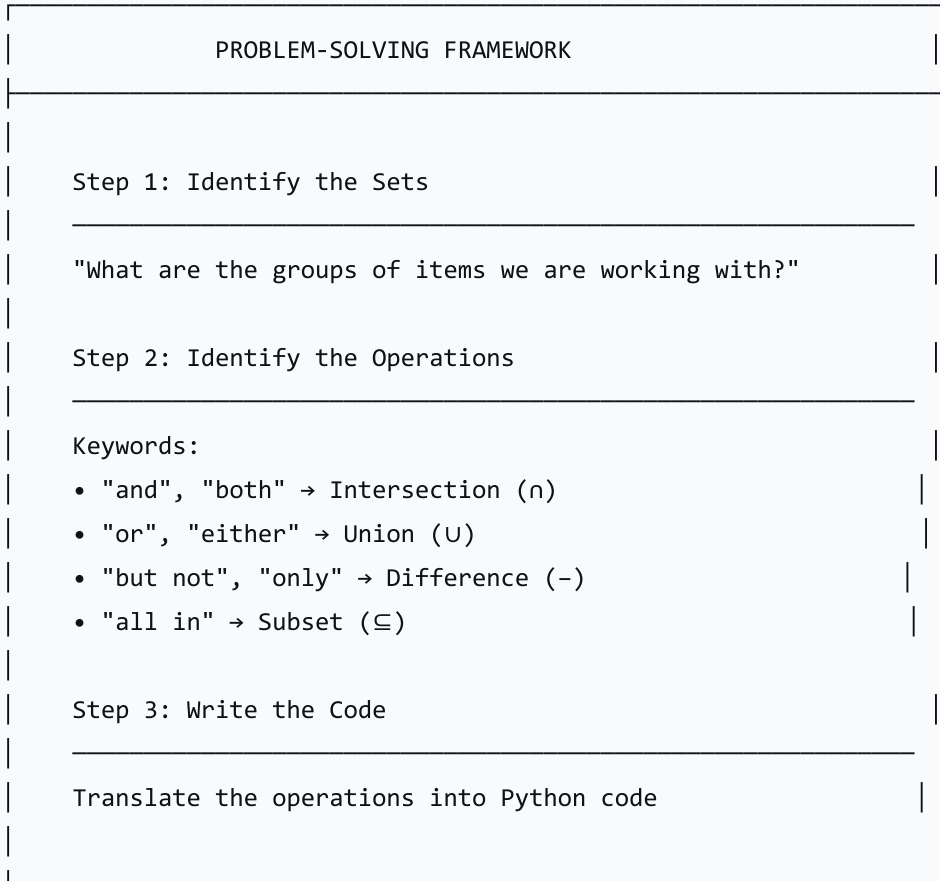
Learning Intentions

By the end of this lesson, students will be able to:

- Analyse a word problem and **model** it using sets
- **Select** the appropriate set operations to solve the problem
- **Combine** multiple set operations to answer complex queries
- **Consolidate** their knowledge of all set characteristics and operations

Problem-Solving Framework

text



Example Problem: Movie Night

Scenario: Two friends want to decide on a movie to watch. We have lists of movies each has seen.

python

```
# Data
alice_movies = {"The Matrix", "Inception", "Interstellar", "The Dark Knight"}
bob_movies = {"Inception", "The Dark Knight", "The Godfather", "Pulp Fiction"}

# 1. Movies both have seen (intersection)
both_seen = alice_movies & bob_movies
print(f"Both have seen: {both_seen}")

# 2. Movies only Alice has seen (difference)
only_alice = alice_movies - bob_movies
```

```

print(f"Only Alice: {only_alice}")

# 3. Movies only Bob has seen (difference)
only_bob = bob_movies - alice_movies
print(f"Only Bob: {only_bob}")

# 4. Movies either has seen (union)
either_seen = alice_movies | bob_movies
print(f"Either has seen: {either_seen}")

```

Example Problem: User Permissions

```

python

# User permission sets
guest_permissions = {"view", "read"}
editor_permissions = {"view", "read", "edit", "publish"}
admin_permissions = {"view", "read", "edit", "publish", "delete", "manage_users"}

# 1. Permissions common to guests and editors (intersection)
common = guest_permissions & editor_permissions
print(f"Common permissions: {common}")

# 2. Permissions editors have that guests don't (difference)
editor_extra = editor_permissions - guest_permissions
print(f"Editor extra permissions: {editor_extra}")

# 3. Permissions only admins have (difference)
admin_only = admin_permissions - editor_permissions
print(f"Admin only permissions: {admin_only}")

# 4. Is guest a subset of editor? (subset check)
is_subset = guest_permissions.issubset(editor_permissions)
print(f"Is guest a subset of editor? {is_subset}")

```

6. Worksheets

Worksheet	Topic
Worksheet 1	Set creation, <code>.add()</code> , <code>in</code> operator
Worksheet 2	Union, intersection, difference (operators and methods)
Worksheet 3	Subsets, supersets, <code>.remove()</code> vs <code>.discard()</code>
Worksheet 4	Problem-solving with sets

7. Summary: Key Takeaways

Concept	Python	Description
Create Set	<code>{1, 2, 3}</code>	Curly braces
Add	<code>.add(x)</code>	Adds element (no effect if exists)
Union	<code>A B</code> or <code>.union()</code>	All elements from both
Intersection	<code>A & B</code> or <code>.intersection()</code>	Common elements
Difference	<code>A - B</code> or <code>.difference()</code>	In A but not B
Subset	<code>A <= B</code> or <code>.issubset()</code>	$A \subseteq B$
Proper Subset	<code>A < B</code>	$A \subset B$
Remove	<code>.remove(x)</code>	Raises <code>KeyError</code> if not found
Discard	<code>.discard(x)</code>	No error if not found
Membership	<code>x in A</code>	Check if element exists

text

KEY REMINDERS

- ✓ Sets are unordered and contain unique elements
- ✓ Union = $A \cup B$ (everything)
- ✓ Intersection = $A \cap B$ (common)
- ✓ Difference = $A - B$ (in A but not B)
- ✓ Subset = $A \subseteq B$ (all A in B)
- ✓ `.remove()` raises error if not found
- ✓ `.discard()` does nothing if not found

"Sets are a powerful ADT for managing collections of unique items—they allow us to perform logical operations like union, intersection, and difference with simple code."